로그인 요청, 진행 표시, 성공 후 이동

- 날짜: 2026-11-02
- 대상: Next.js 16.4.3, React 19.3.0, `lgcns-fe-next`
- 기준은 현재 소스다. 이 글은 `app/(auth)/login/page.tsx`의 서버 함수와, 그 함수를 받는 `LoginForm`을 설명한다.

로그인 버튼을 누르면 브라우저가 Spring 서버를 직접 부르지 않는다. Next 서버 함수가 `http://localhost:8080/api/v1/auth/login`으로 `POST`하고, 응답의 `success`가 참일 때만 `/`로 이동한다. 이동 전에 토큰을 쿠키나 저장소에 넣는 코드는 없다. 응답 본문은 서버 콘솔에만 찍힌다.

1. 파일과 선언

파일	선언	모듈	하는 일
`app/(auth)/login/page.tsx`	`async function handleSubmit` + 본문 첫 줄 `"use server"`	서버 컴포넌트. 파일에 `"use client"`가 없다	`FormData`를 읽고 `fetch`한 뒤, 성공하면 `redirect("/")`
`component/templates/LoginSection.tsx`	`function LoginSection`	서버 컴포넌트	서버 함수를 `LoginForm`에 넘기는 `<section>`
`component/organisms/LoginForm.tsx`	파일 첫 줄 `"use client"`	클라이언트 컴포넌트	입력 상태와 Zod 검사. `next/form`의 `action`에 서버 함수를 넣는다
같은 파일의 `LoginSubmitButton`	`function LoginSubmitButton`	클라이언트 컴포넌트	`useFormStatus()`의 `pending`으로 버튼 문구를 바꾼다
`schema/loginSchema.ts`	`const loginSchema`, `function loginFieldMessage`	클라이언트 그래프	`LoginForm`만 가져온다. 서버 함수는 이 스키마를 부르지 않는다
`types/request/RequestType.ts`	`interface UserPayloadType`	타입	`userId`와 `password`가 `string`이다
`app/(main)/page.tsx`	`function Home`	서버 컴포넌트	`redirect("/")`가 여는 페이지. `HeroSection`과 `AboutSection`을 그린다
`next/navigation`	`import { redirect } from "next/navigation"`	서버 함수 안에서 호출	성공 응답 뒤에만 호출된다

파일	선언	모듈	하는 일
react-dom	import { useFormStatus } from "react-dom"	클라이언트 훅	부모 <form>의 제출 상태를 읽는다
next/form	import Form from "next/form"	클라이언트 모듈	next/dist/client/form.js 첫 줄이 'use client'다

(auth)와 (main)은 라우트 그룹이다. URL에는 없다. 로그인 주소는 /login이고, 성공 후 주소는 /다.

"use server"는 그 함수를 서버 함수로 표시하는 지시문이다. 이 페이지에서는 함수 본문의 첫 문장이다. "use client"는 그 파일이 클라이언트 컴포넌트 그래프의 시작이라는 지시문이다. 두 지시문은 호이스팅 문법이 아니다.

handleSubmit의 async는 ECMAScript의 async 함수다. 호출 결과가 프로미스다. HTML의 <script async>와 같은 속성이 아니다. Next.js에서 서버 함수는 비동기여야 한다. 서버에서 실행되고, 브라우저는 그 함수를 네트워크 요청으로 부르기 때문이다.

2. 로딩 시점

이 로그인 파일은 HTML에 <script src>로 적혀 있지 않다. Next가 서버에서 서버 컴포넌트를 실행하고, "use client"로 시작한 그래프만 브라우저 모듈로 보낸다.

코드	적힌 곳	로딩
<script src="a.js"></script>	HTML. async와 defer가 없음	동기. 파서가 그 자리에서 멈추고, 받고, 실행한 뒤 파싱을 재개한다
<script async src="a.js"></script>	HTML	받는 즉시 실행한다. 여러 스크립트의 실행 순서는 문서 순서가 아니다
<script defer src="a.js"></script>	HTML	문서를 파싱한 뒤, 문서에 나온 순서로 실행한다
<script type="module" src="a.js"></script>	HTML	모듈 그래프. async가 없으면 defer처럼 파싱이 끝난 뒤 실행한다
async function handleSubmit	login/page.tsx	스크립트 로딩 속성이 아니다. 호출될 때 프로미스를 반환하는 함수다. 실행 위치는 Next 서버다
"use client" 파일	LoginForm.tsx	Next가 브라우저용 모듈로 보낸다. 손글씨 <script>는 없다

HTML 명세에서 고전 스크립트에 async도 defer도 없으면 파서를 막는다. defer는 문서 파싱이 끝난 뒤 순서대로 실행한다. type="module"인 외부 스크립트는 async 속성이 없을 때 지연 실행된다. ECMA-262의 async 함수는 그와 별개로, 호출 결과를 Promise로 감싼다.

3. 제출이 서버 함수까지 가는 경로

LoginForm은 화면 값을 useState에 둔다. 제출 때 서버가 읽는 값은 그 상태가 아니다. 입력이 name="userId", name="password"를 갖고 있으므로, 브라우저가 만드는 FormData에 들어간다.

```
<Form action={handleSubmit}>
  <FormField id="userId" name="userId" value={payload.userId} ... />
  <FormField id="password" name="password" value={payload.password} ... />
  <LoginSubmitButton isValid={isValid} />
</Form>
```

Form은 <form>으로 그려지고, action은 서버 함수다. React가 그 폼을 POST로 제출하면 Next가 handleSubmit(formData)를 서버에서 실행한다.

서버 함수는 이렇게 읽는다.

```
const userId = formData.get("userId");
const password = formData.get("password");
```

FormData.get은 그 이름의 첫 값을 반환한다. 값이 없으면 null이다. 반환 타입은 string | File | null 이다. 이 소스는 그 값을 문자열로 좁히지 않고 그대로 JSON에 넣는다. 텍스트 입력이면 문자열이다.

버튼이 꺼져 있는 조건은 클라이언트에만 있다. loginSchema.safeParse(payload).success가 거짓이면 disabled다. 스키마는 두 문자열이 각각 한 글자 이상일 것을 요구한다. 서버 함수는 loginSchema를 부르지 않는다.

4. fetch와 Content-Type

서버 함수의 요청은 이렇다.

```
const response = await fetch(loginApiUrl, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ email: userId, password }),
});
const data = await response.json();
```

JSON.stringify는 객체를 JSON 문자열로 만든다. fetch의 body에 그 문자열을 넣는다.

Fetch 표준은 본문 종류마다 Content-Type 기본값을 정한다. 본문이 문자열이고 호출자가 Content-Type 을 주지 않으면 text/plain;charset=UTF-8이다. Blob은 Blob의 타입, URLSearchParams는 application/x-www-form-urlencoded;charset=UTF-8, FormData는 경계가 있는 multipart/form-data다. 문자열은 JSON이라는 뜻이 아니다. JSON처럼 생긴 글자일 뿐이다.

그래서 헤더 없이 JSON.stringify만 보내면 Spring이 이렇게 거절했다.

HttpMediaTypeNotSupportedException: Content-Type 'text/plain;charset=UTF-8' is not supported

headers에 application/json을 넣으면 그 기본값을 덮어쓴다. MDN의 Fetch 안내도 POST 본문의 형식을 서버에 알리는 헤더로 Content-Type을 들고, JSON 본문 예제에 application/json을 붙인다.

이 fetch는 서버 함수 안에서 돈다. 브라우저 개발자 도구의 문서 요청이 아니라, Next 서버가 localhost:8080으로 보내는 요청이다.

5. 폼 이름 userId와 JSON 키 email

화면 입력 이름은 userId다. UserPayloadType도 userId와 password다. 로그인 API가 받는 JSON 키는 email과 password다. 페이지 주석의 예시는 student@example.com이다.

그래서 본문은 { userId, password }가 아니라 { email: userId, password }다. 폼 필드를 email로 바꾸지는 않았다. 서버 함수가 이름을 바꿔 담는다.

```
body: JSON.stringify({ email: userId, password }),
```

email에 빈 값이 들어가면 API는 success: false와 함께 본문 검증 메시지를 돌려준다. 이전에 보였던 메시지는 must not be blank였다. 그 응답도 redirect까지 가지 않는다.

6. 로그인중 표시

useFormStatus는 부모 <form>이 제출 중인지 알려 준다. 반환값의 pending이 true이면 그 폼의 제출이 끝나지 않은 것이다. 인자는 없다.

이 훅은 <form>을 그리는 함수 안에서 호출하면 그 폼을 보지 못한다. pending은 계속 false다. React 문서는 훅을 폼 안의 자식 컴포넌트에서 호출하라고 한다. 그래서 버튼만 LoginSubmitButton으로 분리했다. LoginForm이 <Form>을 그리고, 그 안에 LoginSubmitButton이 있다.

```
function LoginSubmitButton({ isValid }: { isValid: boolean }) {
  const { pending } = useFormStatus();

  return (
    <Button type="submit" disabled={!isValid || pending} aria-busy={pending}>
      {pending ? "로그인중" : "Login"}
    </Button>
  );
}
```

pending인 동안 문구는 로그인중이고, 버튼은 비활성화되며 aria-busy는 참이다. 제출이 끝나면 문구는 Login으로 돌아온다. pending이 참인 구간에는 서버 함수의 fetch와, 성공 시 이동 처리가 포함된다.

useFormStatus가 돌려주는 다른 값은 이 버튼에서 쓰지 않는다. data는 제출 중인 FormData이고 제출이 없으면 null이다. method는 부모 폼의 HTTP 메서드다. action은 부모 폼의 action에 준 함수다.

7. 성공하면 /로 이동

```
if (data.success) {
  redirect("/");
}
```

redirect는 값을 반환하지 않는다. 내부에서 리다이렉트 오류를 던져 그 이후 코드를 실행하지 않는다. 이 함수에는 try/catch가 없어서, 그 오류를 삼키는 코드도 없다. Next.js 문서는 try/catch를 쓸 때 redirect를 try 밖에 두라고 한다.

서버 함수에서 JavaScript가 있으면 redirect는 클라이언트 쪽 이동을 한다. 서버 액션의 기본 history 동작은 push다. 다른 곳의 기본값은 replace다. 이 호출은 두 번째 인자 type을 주지 않았으므로 서버 함수의 기본값인 push다. JavaScript 없이 폼만 제출되는 점진적 향상에서는 303 응답으로 따라가고, 그 경우 브라우저는 GET으로 연다.

"/"는 app/(main)/page.tsx의 Home이다. 그 페이지는 HeroSection과 AboutSection을 렌더한다.

응답에 accessToken과 refreshToken이 있어도 이 소스는 저장하지 않는다. console.log("data: ", data)로 서버 로그에 남긴 뒤, success만 보고 이동한다. 이동한 메인 페이지가 로그인 상태를 읽도록 연결되어 있지는 않다.

8. 실패하면 로그인 페이지에 남는다

data.success가 거짓이면 redirect는 호출되지 않는다. 서버 함수는 그 메시지로 화면을 갱신하지 않는다. data.message를 보여주는 state는 LoginForm에 없다. 필드의 빨간 문구는 loginFieldMessage가 만든 클라이언트 검사이고, API가 돌려준 문장이 아니다.

비밀번호가 틀린 응답의 한 예는 서버 로그의 success: false, code: 'UNAUTHORIZED'다. 그때 주소는 /login에 남고, 버튼 문구는 제출이 끝난 뒤 Login으로 돌아온다.

9. 공식 문서

- Fetch 표준, 본문에서 Content-Type을 정하는 절차: https://fetch.spec.whatwg.org/#concept-bodyinit-extract
- MDN, Fetch로 헤더를 넣는 방법: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- MDN, FormData.get: https://developer.mozilla.org/en-US/docs/Web/API/FormData/get
- MDN, JSON.stringify: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON/stringify
- ECMA-262, async 함수: https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-async-function-definitions
- ECMA-262, JSON.stringify: https://tc39.es/ecma262/multipage/structured-data.html#sec-json.stringify

- HTML 명세, `script`의 `async`, `defer`, 모듈: https://html.spec.whatwg.org/multipage/scripting.html#the-script-element
- React, `useFormStatus`: https://react.dev/reference/react-dom/hooks/useFormStatus
- Next.js, `redirect`: https://nextjs.org/docs/app/api-reference/functions/redirect
- Next.js, 서버 함수로 데이터 바꾸기: https://nextjs.org/docs/app/getting-started/updating-data